

TIC-TAC-TOE AGENT :

```
import random

board = [' ' for _ in range(9)]

HUMAN = 'X'

COMPUTER = 'O'

WIN_CONDITIONS = [
    [0, 1, 2], [3, 4, 5], [6, 7, 8], # rows
    [0, 3, 6], [1, 4, 7], [2, 5, 8], # columns
    [0, 4, 8], [2, 4, 6]             # diagonals
]

def print_board():
    print()
    for row in range(3):
        r = board[row*3:row*3+3]
        print(' ' + ' | '.join(r))
        if row < 2:
            print('-----')
    print()

def check_winner(player):
    for combo in WIN_CONDITIONS:
        if all(board[i] == player for i in combo):
            return True
    return False
```

```
def is_board_full():
```

```
    return ' ' not in board
```

```
def get_human_move():
```

```
    while True:
```

```
        try:
```

```
            move = int(input("Your turn (enter position 1-9): ")) - 1
```

```
            if move < 0 or move > 8:
```

```
                print("Invalid position! Choose between 1 and 9.")
```

```
            elif board[move] != ' ':
```

```
                print("Cell already taken! Choose another position.")
```

```
            else:
```

```
                return move
```

```
        except ValueError:
```

```
            print("Please enter a valid number.")
```

```
def find_winning_move(player):
```

```
    # Check if 'player' can win in the next move; return that position
```

```
    for combo in WIN_CONDITIONS:
```

```
        values = [board[i] for i in combo]
```

```
        if values.count(player) == 2 and values.count(' ') == 1:
```

```
            return combo[values.index(' ')]
```

```
    return None
```

```
def get_computer_move():
```

```
    # 1. Try to win
```

```
    move = find_winning_move(COMPUTER)
```

```
if move is not None:
```

```
    return move
```

```
# 2. Block human's winning move
```

```
move = find_winning_move(HUMAN)
```

```
if move is not None:
```

```
    return move
```

```
# 3. Take center if free
```

```
if board[4] == '':
```

```
    return 4
```

```
# 4. Take a free corner
```

```
corners = [0, 2, 6, 8]
```

```
free_corners = [c for c in corners if board[c] == '']
```

```
if free_corners:
```

```
    return random.choice(free_corners)
```

```
# 5. Take any free edge
```

```
edges = [1, 3, 5, 7]
```

```
free_edges = [e for e in edges if board[e] == '']
```

```
if free_edges:
```

```
    return random.choice(free_edges)
```

```
def play_game():
```

```
    print("Welcome to Tic-Tac-Toe! You are 'X', Computer is 'O'.")
```

```
    print_board()
```

```
    current_player = HUMAN
```

```

while True:

    if current_player == HUMAN:

        move = get_human_move()

    else:

        move = get_computer_move()

        print(f"Computer chooses position {move + 1}")

    board[move] = current_player

    print_board()

    if check_winner(current_player):

        winner = "You" if current_player == HUMAN else "Computer"

        print(f"{winner} win{'s' if winner == 'Computer' else ''}! 🎉")

        break

    if is_board_full():

        print("It's a Draw!")

        break

    current_player = COMPUTER if current_player == HUMAN else HUMAN

if __name__ == "__main__":

    play_game()

```

OUTPUT :

Welcome to Tic-Tac-Toe! You are 'X', Computer is 'O'.

```
| |  
-----  
| |  
-----  
| |
```

Your turn (enter position 1-9): 1

```
X | |  
-----  
| |  
-----  
| |
```

Computer chooses position 5

```
X | |  
-----  
| O |  
-----  
| |
```

Your turn (enter position 1-9): 9

```
X | |  
-----  
| O |  
-----  
| | X
```

Computer chooses position 7

```
X | |  
-----  
| O |  
-----  
O | | X
```

Your turn (enter position 1-9): 3

```
X | | X  
-----  
| O |  
-----  
O | | X
```

Computer chooses position 2

```
X | O | X  
-----  
| O |  
-----  
O | | X
```

Your turn (enter position 1-9): 6

```
X | O | X  
-----  
| O | X  
-----  
O | | X
```

You win! 🎉

VACCUM CLEANER AGENT:

```
def goal_achieved(environment):  
    return all(status == 'Clean' for status in environment.values())  
  
def goal_based_agent(location, environment):  
    if environment[location] == 'Dirty':  
        return 'Suck'  
    elif location == 'A':  
        return 'Right'  
    else:  
        return 'Left'  
  
def run_goal_based_agent(environment):  
    print("\n--- GOAL-BASED AGENT ---")  
    location = 'A'  
    step = 0  
  
    while not goal_achieved(environment):  
        step += 1  
        status = environment[location]  
        print(f"Step {step}: Location={location}, Status={status}")  
  
        action = goal_based_agent(location, environment)  
        print(f" Action Taken: {action}")  
  
        if action == 'Suck':  
            environment[location] = 'Clean'  
        elif action == 'Right':  
            location = 'B'
```

```
elif action == 'Left':
```

```
    location = 'A'
```

```
print(f"Goal Achieved in {step} steps! Environment is fully clean.")
```

```
print("Final Environment State:", environment)
```

```
def get_status_input(location):
```

```
    while True:
```

```
        status = input(f"Enter status for Location {location} (Clean/Dirty): ").strip().capitalize()
```

```
        if status in ('Clean', 'Dirty'):
```

```
            return status
```

```
        print("Invalid input! Please enter 'Clean' or 'Dirty'.")
```

```
if __name__ == "__main__":
```

```
    print("=== Vacuum Cleaner Agent (Goal-Based) ===")
```

```
    env = {
```

```
        'A': get_status_input('A'),
```

```
        'B': get_status_input('B')
```

```
    }
```

```
    print("\nInitial Environment for Goal-Based Agent:", env)
```

```
    run_goal_based_agent(env)
```

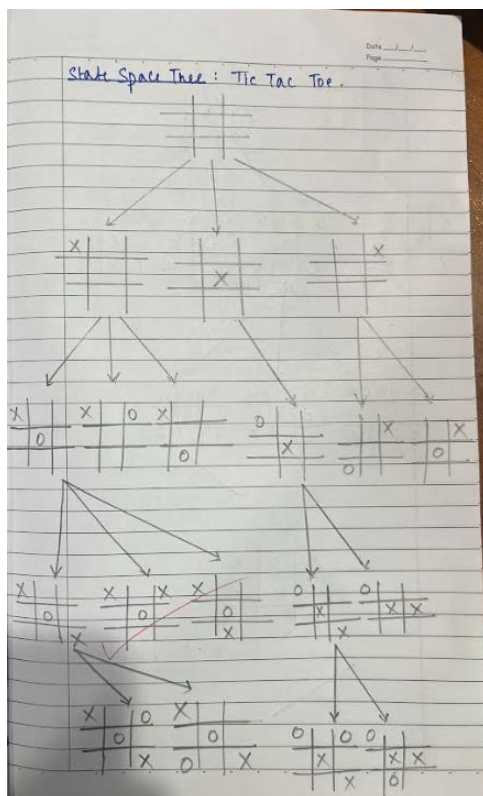
OUTPUT :

```
=== Vacuum Cleaner Agent (Goal-Based) ===
Enter status for Location A (Clean/Dirty): dirty
Enter status for Location B (Clean/Dirty): dirty

Initial Environment for Goal-Based Agent: {'A': 'Dirty', 'B': 'Dirty'}

--- GOAL-BASED AGENT ---
Step 1: Location=A, Status=Dirty
Action Taken: Suck
Step 2: Location=A, Status=Clean
Action Taken: Right
Step 3: Location=B, Status=Dirty
Action Taken: Suck
Goal Achieved in 3 steps! Environment is fully clean.
Final Environment State: {'A': 'Clean', 'B': 'Clean'}
```

NOTEBOOK :



Date: / /
Page:

Tic Tac Toe Agent

OUTPUT:
Welcome to Tic Tac Toe! You are 'X', Computer is 'O'.

Your turn (enter position 1-9): 1

X		

Computer chooses position 5

X		
	O	

Your turn (enter position 1-9): 9

X		
	O	
		X

Computer chooses position 7

X		
	O	
O		X

Your turn (enter position 1-9): 3

X		X
	O	
O		X

Computer chooses position 2

X	O	X
	O	
O		X

Your turn (enter position 1-9): 6

X	O	X
	O	X
O		X

You Win!

Vacuum Agent

OUTPUT:
— Vacuum Cleaner Agent —
Enter Status for location A (Clean/Dirty):
dirty
Enter Status for location B (Clean/Dirty):
dirty

Initial Environment: {A: 'Dirty', B: 'Dirty'}
 step 1: location = A, Status = Dirty
 Action taken = Suck
 step 2: location = A, Status = Clean
 Action taken = Right
 step 3: location = B, Status = Dirty
 Action taken = Suck
 Goal achieved in 3 steps! Environment is clean

